

Iterazione 1

Durante l' iterazione 1, ci si è occupati di implementare il caso d'uso UC1 con relativo caso d'uso d'avviamento e relativi test.

Riporto il UC1 sviluppato:

UC1: Cerca itinerario

Nome del caso d'uso	Cerca itinerario
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: vuole cercare degli itinerari data posizione di partenza e posizione di destinazione
Pre-condizioni	L' applicazione è in possesso dei dati relativi al traffico
Garanzie di successo	L' Utente visualizza gli itinerari basati sulle posizioni da lui fornite
Scenario principale di successo	1. L' Utente vuole cercare un itinerario date posizioni di partenza e destinazione 2. L' Utente sceglie l'operazione "Cerca Itinerario" 3. Il Sistema fornisce una lista di città di partenza 4. L'Utente inserisce il codice della città di partenza 5. Il Sistema fornisce una lista di città di destinazione 6. L'Utente inserisce il codice della città di destinazione 7. Il Sistema sulla base dei dati immessi, fornisce le informazioni relative agli itinerari 8. L' Utente ha visualizzato il risultato 9. L' Utente sceglie un Itinerario da visualizzare in dettaglio 10. Il Sistema restituisce tutte le fermate relative all'Itinerario scelto dall'Utente I passi 9 e 10 vengono ripetuti finché serve 11. Il Sistema riporta l'Utente al Menù principale

Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L' Utente inserisce un codice di città di partenza non valido 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 4 inserendo un codice alternativo 6a: L' Utente inserisce un codice codice di città di arrivo non valido 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 6 inserendo un codice alternativo 6b: L' Utente inserisce lo stesso codice di città in partenza e destinazione 1. Il Sistema genera un messaggio di errore 2. L'Utente ripete il passo 6 inserendo un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al numero di utenze
Varie	

Modello di dominio

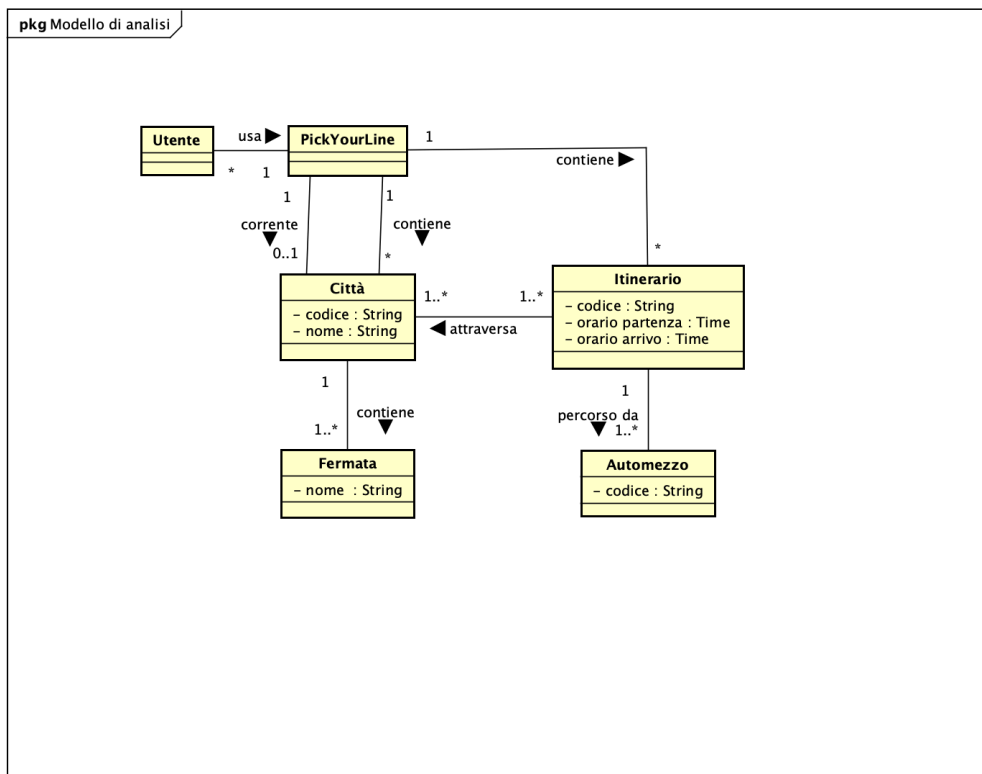
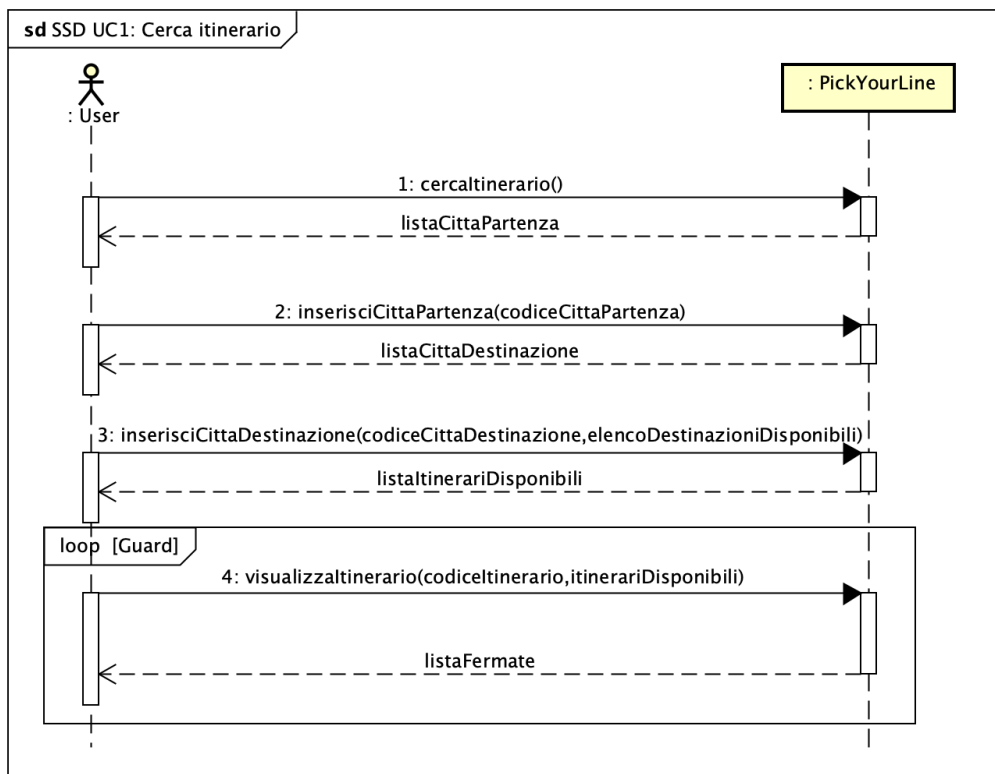


Diagramma di sequenza di sistema



Contratti delle operazioni

Operazione	cercaltinerario()
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	Esiste la lista delle città e dei relativi itinerari
Post-condizioni	Viene recuperata la lista contenente le istanze delle Città di partenza

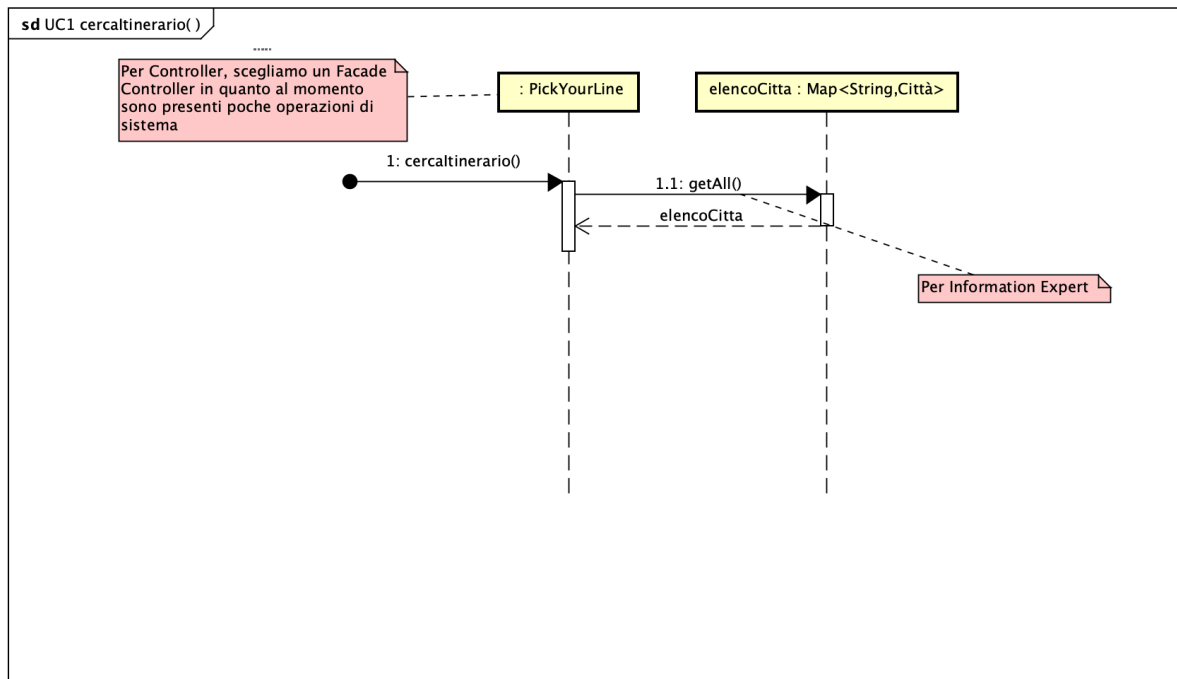
Operazione	inserisciCittaPartenza(codiceCittaPartenza)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario
Post-condizioni	-Viene recuperata l'istanza della città di partenza cp scelta dall'utente -Viene associata l'istanza cp all'istanza di PickYourLine -Viene recuperata la lista contenente le istanze delle Città di destinazione

Operazione	inserisciCittaDestinazione(codiceCittaDestinazione)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario
Post-condizioni	Viene recuperata la lista contenente le istanze di Itinerario

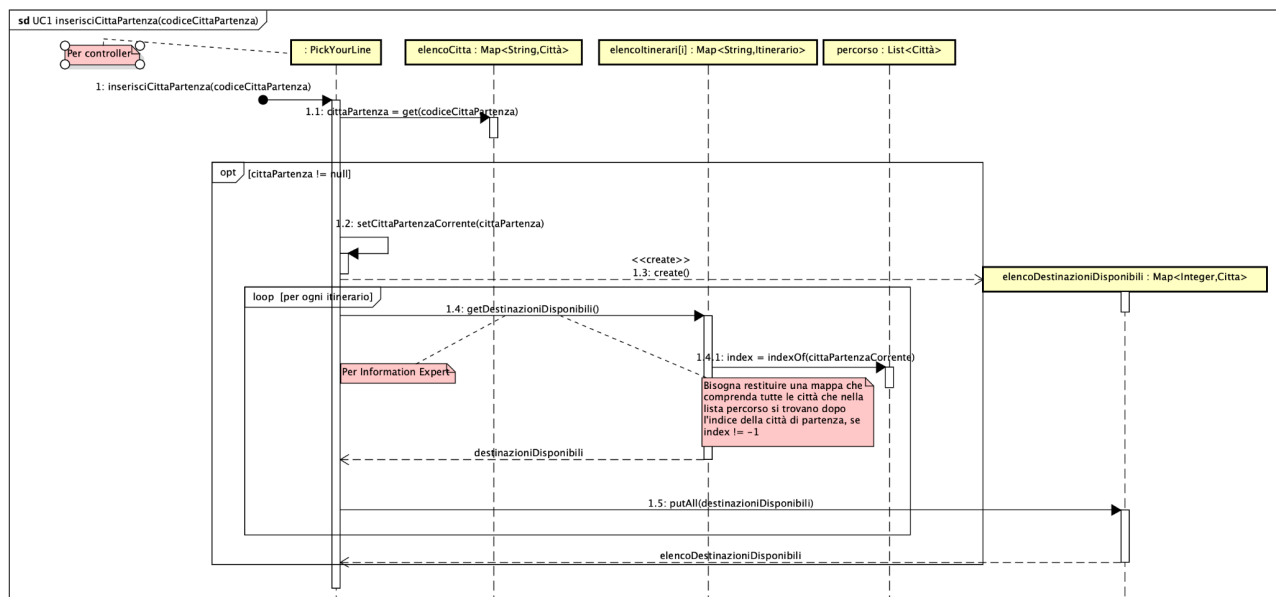
Operazione	visualizzaltinerario(codiceltinerario)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario
Post-condizioni	Viene recuperata la lista contenente le istanze di Fermate, relativa all' Itinerario scelto

Diagrammi di sequenza

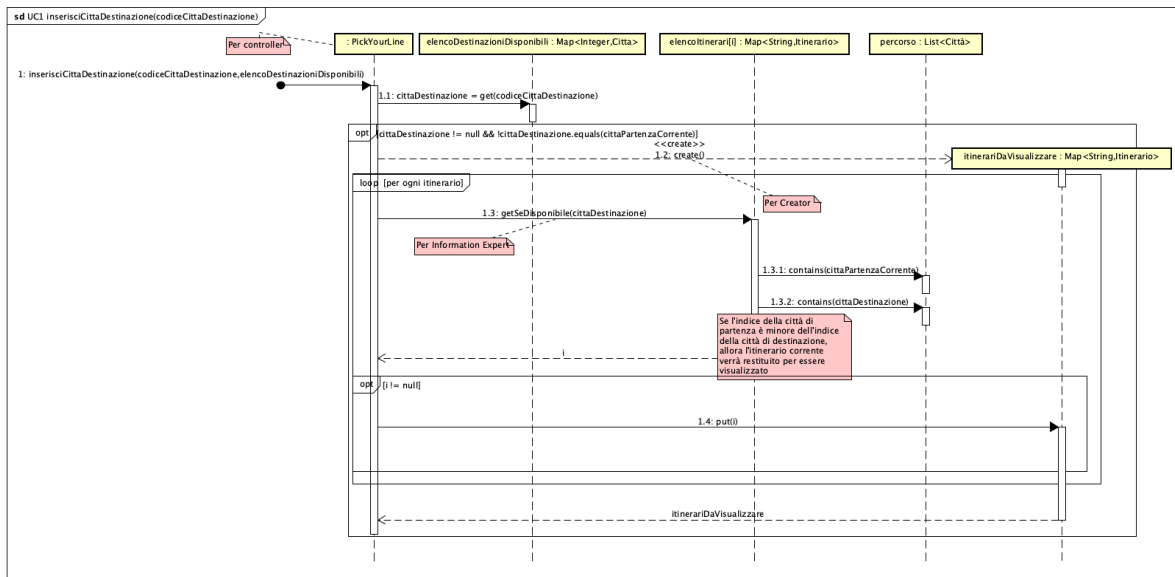
- cercaltinerario



- inserisciCittaDiPartenza



- InserisciCittaDestinazione



- visualizzaItinerario

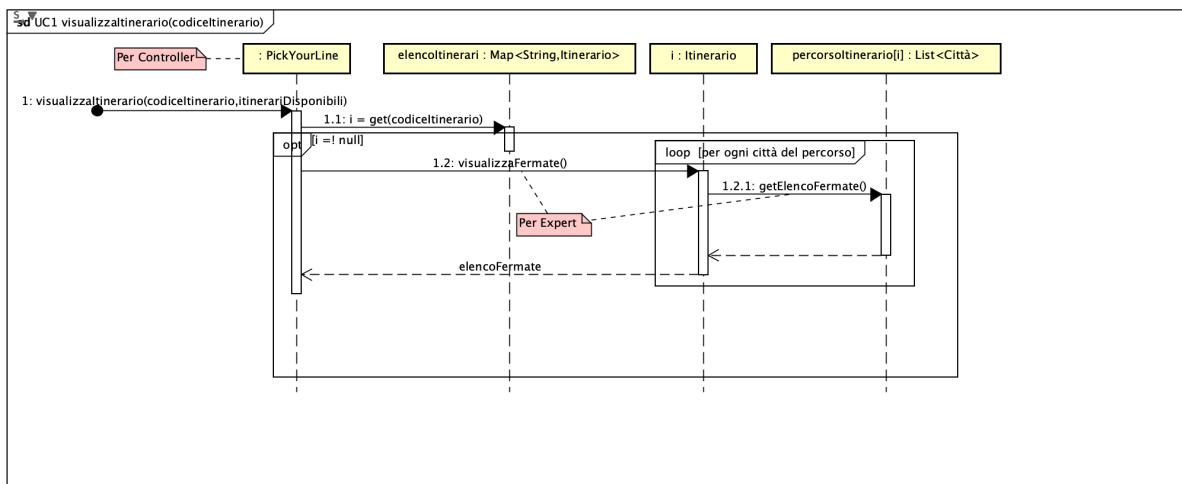
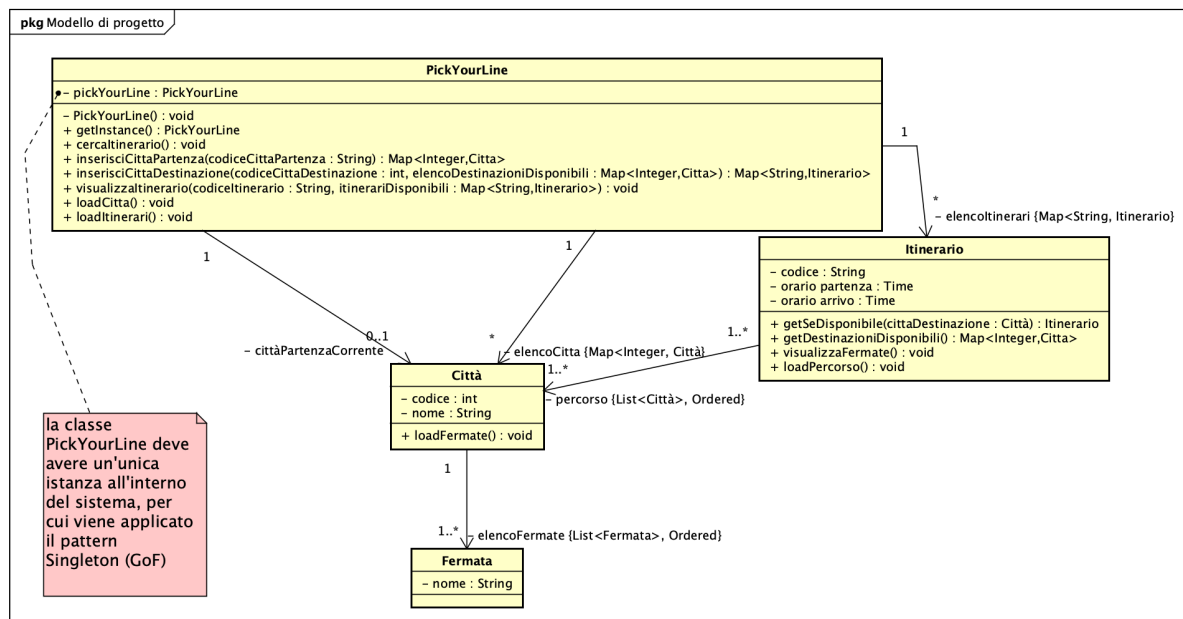
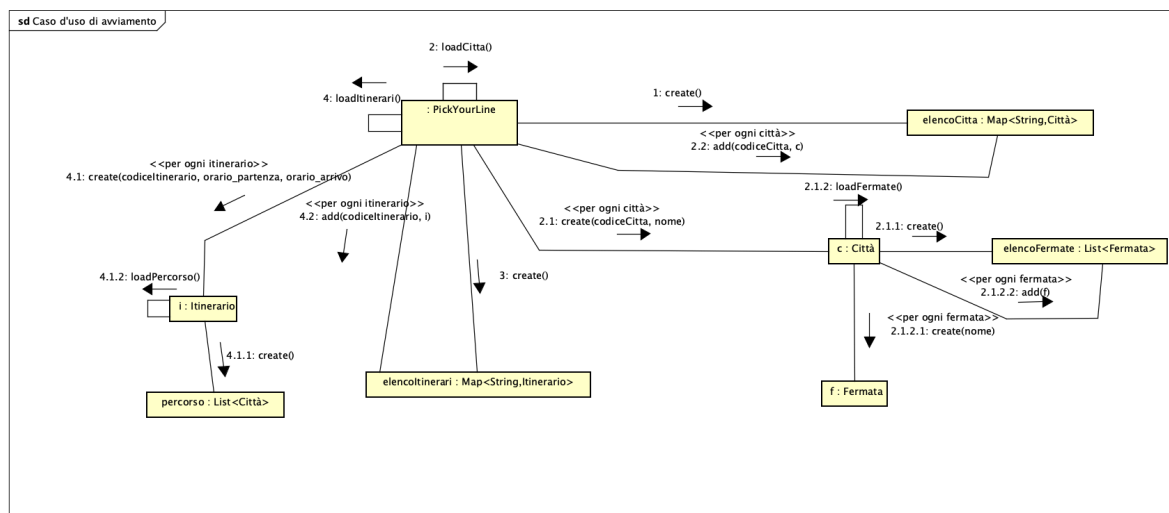


Diagramma delle classi di progetto



Caso d'uso d'avviamento



Testing

Per il caso d'uso UC1 abbiamo ritenuto utile testare i seguenti metodi:

testInserisciCittaDiPartenza()

Scopo: Verificare che inserendo il codice di una città di partenza, venga mostrato l'elenco delle destinazioni corretto

Realizzazione: Creiamo una mappa di città attese e la confrontiamo con il risultato dell'operazione, data una specifica città di partenza

testInserisciCittaDestinazione_Successo()

Scopo: Verificare che inserendo il codice di una città di destinazione, venga restituito il corretto elenco di itinerari disponibili

Realizzazione: Inseriamo nell'elenco itinerari degli itinerari, verifichiamo che gli itinerari effettivamente disponibili siano stati aggiunti all'elenco

testInserisciCittaDestinazione_StessaCitta()

Scopo: Verificare che venga sollevata un'eccezione nel caso in cui città di destinazione e di partenza siano uguali

Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserisciCittaDestinazione_CodiceNonValido()

Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un codice di città destinazione non valido

Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserisciCittaDiPartenzaInesistente()

Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un codice di città partenza non esistente

Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaItinerarioValido()

Scopo: Verificare che venga sollevata un'eccezione nel caso in cui l'itinerario non sia presente tra gli itinerari validi

Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testGetSeDisponibile_Positivo()

Scopo: Verificare se l'itinerario è disponibile data una città di partenza.

Realizzazione: Viene settata la città corrente, viene verificato il metodo con una città di destinazione differente.

testGetSeDisponibile_Negativo()

Scopo: Verificare se l'itinerario è disponibile data una città di partenza per percorsi inversi

Realizzazione: Viene settata la città corrente, viene verificato il metodo con una città di destinazione differente.

testGetSeDisponibile_CittaNonNelPercorso()

Scopo: Verificare se la città è disponibile in un itinerario nel caso in cui non è presente nel percorso

Realizzazione: viene creata una città, viene verificato il metodo data codesta città.

testGetDestinazioniDisponibili_ContienePiuDestinazioni()

Scopo: Verificare che l'elenco delle destinazioni disponibili contenga più di una destinazione

Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_ContieneUnaDestinazioni()

Scopo: Verificare che l'elenco delle destinazioni disponibili contenga una sola destinazione

Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_ContieneNessunaDestinazione()

Scopo: Verificare che l'elenco delle destinazioni disponibili non contenga nessuna destinazione

Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_CittaNonInPercorso()

Scopo: Verificare che data una città, essa non appartenendo al percorso, viene restituita un elenco di destinazioni vuoto

Realizzazione: Viene settata una città di partenza, viene verificato il metodo e valutato se il risultato sia una struttura dati vuota

